UNIVERSIDAD NACIONAL AUTÓNOMA DE HONDURAS

FACULTAD DE INGENIERIA

Manual Técnico Sistema Gestión de Proyectos de Software

Ing. Harold Coello

Ingeniería del Software IS-802

Sección 1800

Integrantes:

José Daniel Núñez Oyuela 20221001249

David Fernando Parada Matute 20211002496

Ronny Josué Posadas Díaz 20161002484

Ciudad Universitaria

Tegucigalpa M.D.C 11 de abril del 2026

Introducción

El presente manual técnico detalla la arquitectura, el stack tecnológico y los procedimientos de despliegue del Sistema de Gestión de Proyectos para Entornos Académicos. Este sistema nace como una alternativa soberana para la comunidad estudiantil de la Universidad Nacional Autónoma de Honduras (UNAH), permitiendo la gestión de proyectos bajo metodologías ágiles (Scrum/Kanban) sin las restricciones de plataformas comerciales.

Arquitectónicamente, el sistema se fundamenta en un modelo de Arquitectura en Capas (N-Tier), garantizando una separación clara entre la interfaz de usuario, la lógica de negocio y la persistencia. La solución integra un robusto backend desarrollado en Java con Spring Boot, un frontend reactivo en React y una persistencia de datos en MySQL. La portabilidad está asegurada mediante la contenedorización con Docker, facilitando un despliegue consistente tanto en entornos locales como en la nube (Microsoft Azure para servicios y Vercel para la interfaz).

Este documento actúa como la guía definitiva para desarrolladores y evaluadores, proporcionando las instrucciones necesarias para replicar el entorno de desarrollo, comprender el modelo de datos y asegurar la escalabilidad futura del sistema.

Backend (API REST)

Acceso al repositorio público: https://github.com/Josue-Dz/Proyecto_Ingenieria_Software.git

- **Stack Tecnológico**

El servidor está construido bajo una arquitectura robusta que garantiza seguridad y escalabilidad.

- **Lenguaje:** Java 17 (Eclipse Temurin)
- **Framework:** Spring Boot 4.0.4
- **Gestor de Dependencias:** Maven
- **Seguridad:** Spring Security + JSON Web Token (JWT)
- **Comunicación:** WebSockets para notificaciones en tiempo real
- **Base de Datos:** MySQL 8.0
- **Documentación/Salud:** Spring Boot Actuator

- **Dependencias principales:**

El archivo pom.xml integra librerías de alto nivel para facilitar el desarrollo y la estabilidad:

- **Lombok:** Para la reducción de código repetitivo (Boilerplate).
- **JPA/Hibernate:** Para el mapeo objeto-relacional (ORM) con actualización automática de esquemas (ddl-auto=update).
- **JJWT (v0.13.0):** Implementación de la seguridad basada en tokens para proteger los endpoints.
- **Validation:** Para asegurar la integridad de los datos recibidos en las peticiones (DTOS).

- **Configuración del entorno local:**

A. Requisitos Previos

- **JDK 17** instalado y configurado en las variables de entorno.
- **Docker Desktop** (para la base de datos).
- **IDE:** IntelliJ IDEA, Eclipse o VS Code.

B. Configuración de Base de Datos

El sistema utiliza **MySQL**. Se puede levantar la base de datos rápidamente con el archivo docker-compose.yml, ejecutando estos comandos en consola:

Desde la raíz del proyecto

docker-compose up –d

Nota: *El contenedor creará automáticamente la base de datos my_project_db_ingenieria y estará disponible en el puerto* ***3306****.*

C. Propiedades del Sistema (application.properties)

- **Puerto del servidor:** 8080
- **URL de Conexión:** jdbc: mysql://localhost:3306/my_project_db_ingenieria
- **Auto-DDL:** Activado (update), lo que significa que Spring Boot creará las tablas automáticamente al iniciar según las clases @Entity.

❖ **Ejecución del Proyecto**

Opción 1: Mediante la consola se puede inicializar mediante el comando:

mvn spring-boot:run

Opción 2: Ejecución mediante Docker (Contenerizado)

El proyecto utiliza un **Multi-stage build** para optimizar el peso de la imagen final.

1. **Construcción de la imagen:**

   docker build -t codeflow-backend.

2. **Ejecución del contenedor:**

   docker run -p 8080:8080 codeflow-backend

❖ **Arquitectura del codigo**

El backend sigue el patrón de **Arquitectura en Capas**:

- **Capa de Entidades (Entity):** Mapeo de tablas de BD (con persistencia JPA).
- **Capa de Repositorios (Repository):** Interfaces que heredan de JpaRepository para el acceso a datos.
- **Capa de Servicios (Service):** Contiene la lógica de negocio y validaciones.

- **Capa de Controladores (Controller):** Endpoints REST que gestionan las peticiones HTTP y respuestas JSON.
- **Capa de Seguridad (Security):** Filtros JWT para validar el acceso a rutas protegidas.

Frontend (Interfaz de Usuario)

❖ Stack Tecnológico

La interfaz ha sido diseñada para ser reactiva, rápida y visualmente atractiva, optimizada para la gestión de tareas en tiempo real.

- **Framework:** React 19.2.0
- **Herramienta de Construcción (Build Tool):** Vite 7.3
- **Estilos:** Tailwind CSS 4.2
- **Gestión de Rutas:** React Router DOM 7.1
- **Gráficos y Métricas:** Recharts 3.8 (Utilizado para los reportes solicitados)

❖ Dependencias Críticas y Funcionalidades

El frontend integra librerías especializadas para cumplir con los requerimientos del proyecto:

- **@dnd-kit (Abstract, Helpers, React):** Proporciona la lógica de *Drag and Drop* para mover las tarjetas entre columnas de manera fluida.
- **Axios:** Cliente HTTP para la comunicación con la API REST de Spring Boot.
- **SockJS + StompJS:** Implementación del lado del cliente para las notificaciones en tiempo real vía WebSockets.
- **Tailwind CSS:** Framework de utilidades para el diseño adaptable (Responsive) y la implementación del **Modo Oscuro**.

❖ Configuración del Entorno (Local)

A. Requisitos Previos

- **Node.js:** Versión 18.0 o superior recomendada.
- **NPM o PNPM:** Gestor de paquetes.

B. Instalación

1. Clonar el repositorio y entrar a la carpeta del frontend:

   cd projectflow-frontend

2. Instalar las dependencias:

   npm install

C. Variables de Entorno

Crear un archivo **.env** en la raíz del frontend para apuntar al backend:

VITE_PUBLIC_BACKEND_URL=http://localhost:8080/api

- **Ejecución y despliegue**

Modo Desarrollo: Para levantar el servidor de desarrollo utilizar el comando

npm run dev

El sistema estará disponible por defecto en: http://localhost:5173

Utilizar el comando ***npm run build*** para generar los archivos estáticos optimizados que se suben a Vercel.

- **Arquitectura del Frontend**

El proyecto sigue una estructura modular para facilitar el mantenimiento:

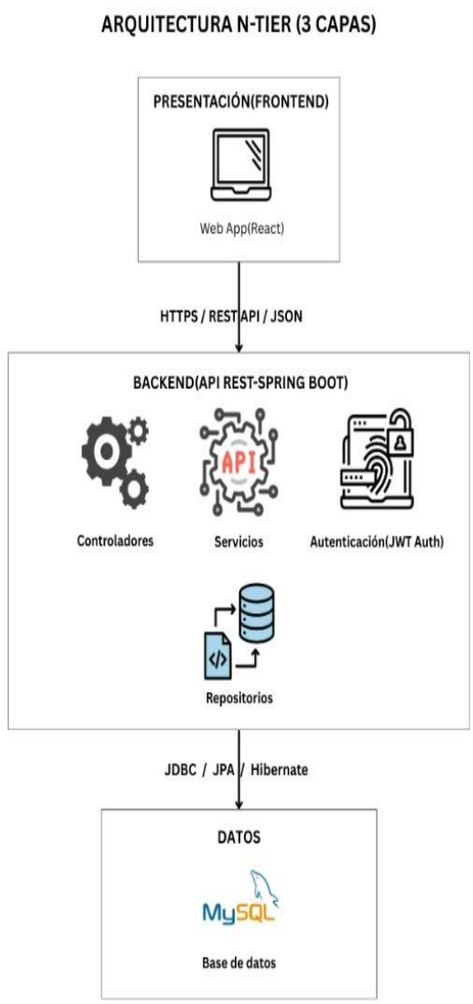
1. **/components:** Elementos reutilizables (Botones, Inputs, Cards).
2. **/pages:** Pantallas principales (Dashboard, Tablero Kanban, Login).
3. **/services:** Funciones de Axios para llamadas a la API (Ej. **boardService.js**).
4. **/context:** Manejo del estado global (Autenticación y Temas).
5. **/hooks:** Lógica personalizada para el manejo del socket y el arrastre de tareas.

- **Manejo de Errores Comunes**

Como se especificó en la fase de desarrollo, el sistema incluye validaciones para:

- **Nulidad en Columnas:** Prevención de errores tipo ***Cannot read properties of null*** mediante encadenamiento opcional (columna?.idColumna).
- **Reconexión de Sockets:** Gestión automática cuando el servidor de notificaciones se desconecta.

Esquema de arquitectura



Conclusiones

El sistema de Gestión de Proyectos para Entornos Académicos representa una solución integral que equilibra la complejidad técnica con la usabilidad. A través de la implementación de una arquitectura en capas y el uso de un stack moderno (React 19 + Spring Boot 4), se ha logrado desarrollar una plataforma capaz de gestionar flujos de trabajo ágiles de manera eficiente, segura y colaborativa.

Podemos identificar logros técnicos que se tuvieron en el proyecto, los cuales son:

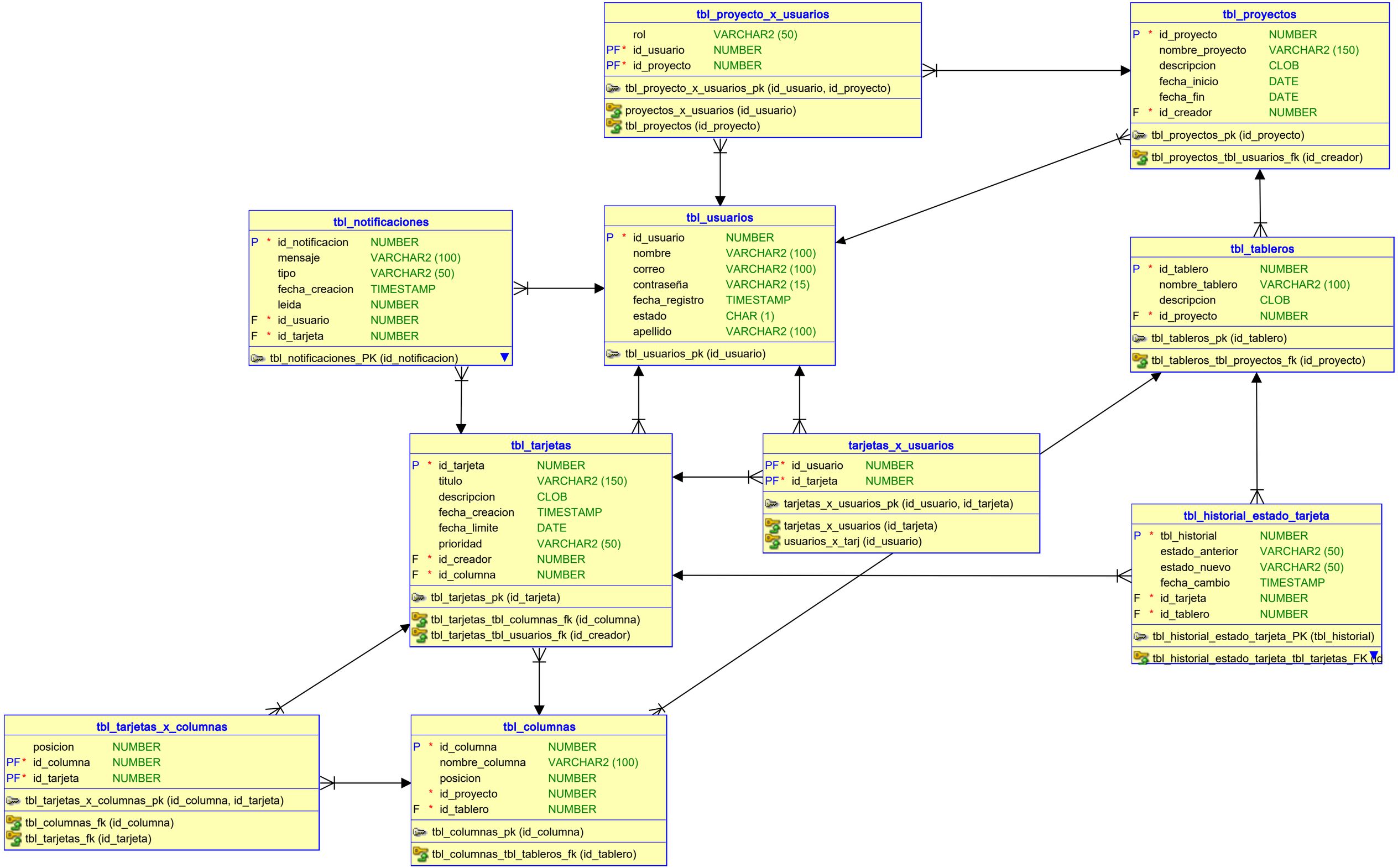
- **Interoperabilidad:** Se estableció una comunicación fluida y segura entre el frontend y el backend mediante el uso de JWT y WebSockets para notificaciones en tiempo real.
- **Escalabilidad:** Gracias a la contenedorización con **Docker**, el sistema está preparado para ser desplegado en diversos entornos, garantizando que la configuración sea idéntica tanto en local como en la nube.
- **Calidad de Software:** La creación de los 8 diagramas UML y la implementación de pruebas unitarias aseguran que el sistema no solo funciona, sino que posee una base documental sólida para futuras extensiones o mantenimientos.

Si bien el sistema cumple con los requerimientos de un Producto Mínimo Viable (MVP) funcional, se identifican las siguientes líneas de evolución:

- **Módulo de Analítica Avanzada:** Integración de reportes más profundos sobre la productividad de cada miembro del equipo.
- **Optimización de Notificaciones:** Implementación de notificaciones vía correo electrónico institucional o aplicaciones móviles.
- **Mejora en la Gestión de Archivos:** Permitir adjuntar documentación técnica directamente en las tarjetas del tablero.

Este proyecto ha permitido al equipo **CodeFlow** consolidar conocimientos en el ciclo de vida completo del desarrollo de software. Desde el modelado de la base de datos hasta el despliegue final en Azure, el proceso ha reforzado la importancia de las metodologías ágiles y la documentación técnica rigurosa como pilares para el éxito de cualquier iniciativa tecnológica en el ámbito profesional.

Al final de este documento se adjuntará el esquema de base de datos y los diagramas UML.


tbl_proyecto_x_usuarios
rol VARCHAR2 (50)
PF * id_usuario NUMBER
PF * id_proyecto NUMBER
tbl_proyecto_x_usuarios_pk (id_usuario, id_proyecto)
proyectos_x_usuarios (id_usuario)
tbl_proyectos (id_proyecto)
tbl_proyectos
P * id_proyecto NUMBER
nombre_proyecto VARCHAR2 (150)
descripcion CLOB
fecha_inicio DATE
fecha_fin DATE
F * id_creador NUMBER
tbl_proyectos_pk (id_proyecto)
tbl_proyectos_tbl_usuarios_fk (id_creador)
tbl_notificaciones
P * id_notificacion NUMBER
mensaje VARCHAR2 (100)
tipo VARCHAR2 (50)
fecha_creacion TIMESTAMP
leida NUMBER
F * id_usuario NUMBER
F * id_tarjeta NUMBER
tbl_notificaciones_PK (id_notificacion)
tbl_usuarios
P * id_usuario NUMBER
nombre VARCHAR2 (100)
correo VARCHAR2 (100)
contraseña VARCHAR2 (15)
fecha_registro TIMESTAMP
estado CHAR (1)
apellido VARCHAR2 (100)
tbl_usuarios_pk (id_usuario)
tbl_tableros
P * id_tablero NUMBER
nombre_tablero VARCHAR2 (100)
descripcion CLOB
F * id_proyecto NUMBER
tbl_tableros_pk (id_tablero)
tbl_tableros_tbl_proyectos_fk (id_proyecto)
tbl_tarjetas
P * id_tarjeta NUMBER
titulo VARCHAR2 (150)
descripcion CLOB
fecha_creacion TIMESTAMP
fecha_limite DATE
prioridad VARCHAR2 (50)
F * id_creador NUMBER
F * id_columna NUMBER
tbl_tarjetas_pk (id_tarjeta)
tbl_tarjetas_tbl_columnas_fk (id_columna)
tbl_tarjetas_tbl_usuarios_fk (id_creador)
tarjetas_x_usuarios
PF * id_usuario NUMBER
PF * id_tarjeta NUMBER
tarjetas_x_usuarios_pk (id_usuario, id_tarjeta)
tarjetas_x_usuarios (id_tarjeta)
usuarios_x_tarj (id_usuario)
tbl_historial_estado_tarjeta
P * tbl_historial NUMBER
estado_anterior VARCHAR2 (50)
estado_nuevo VARCHAR2 (50)
fecha_cambio TIMESTAMP
F * id_tarjeta NUMBER
F * id_tablero NUMBER
tbl_historial_estado_tarjeta_PK (tbl_historial)
tbl_historial_estado_tarjeta_tbl_tarjetas_FK (id
tbl_tarjetas_x_columnas
posicion NUMBER
PF * id_columna NUMBER
PF * id_tarjeta NUMBER
tbl_tarjetas_x_columnas_pk (id_columna, id_tarjeta)
tbl_columnas_fk (id_columna)
tbl_tarjetas_fk (id_tarjeta)
tbl_columnas
P * id_columna NUMBER
nombre_columna VARCHAR2 (100)
posicion NUMBER
* id_proyecto NUMBER
F * id_tablero NUMBER
tbl_columnas_pk (id_columna)
tbl_columnas_tbl_tableros_fk (id_tablero)

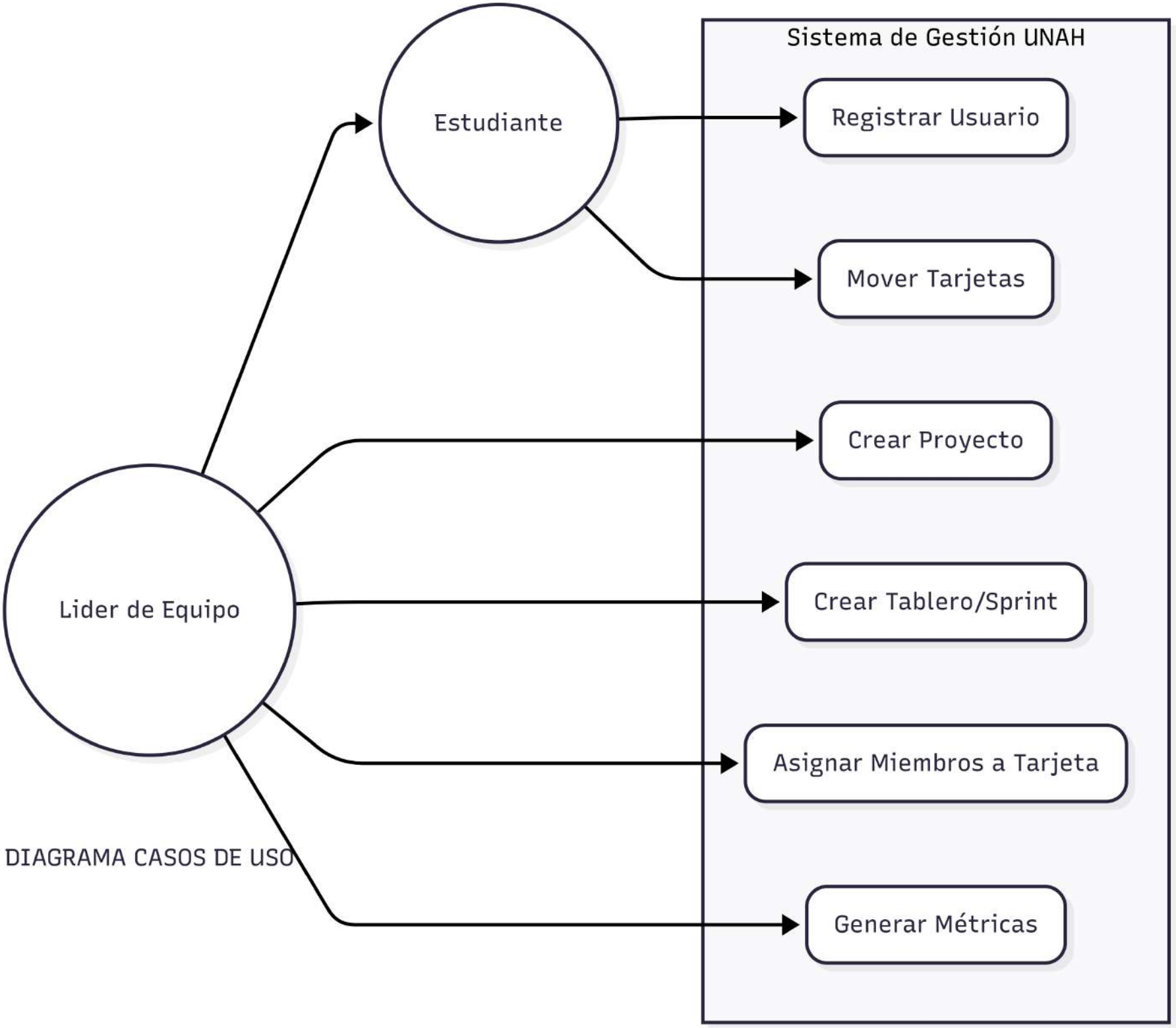
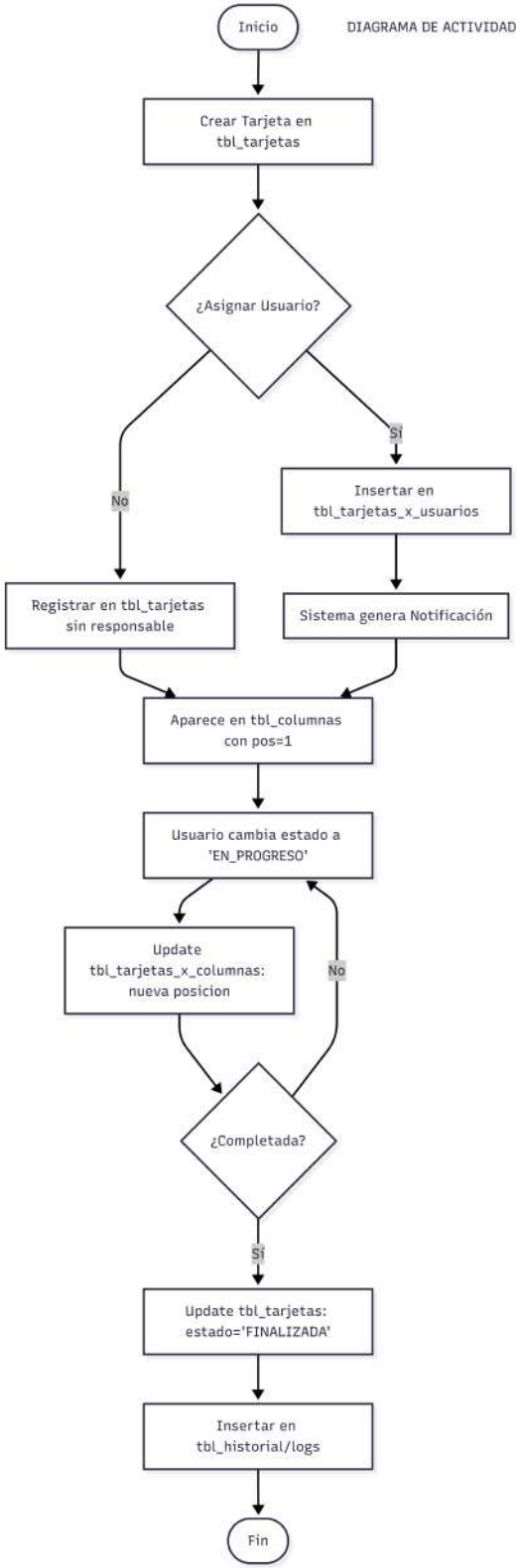


DIAGRAMA CASOS DE USO

DIAGRAMA DE ACTIVIDAD



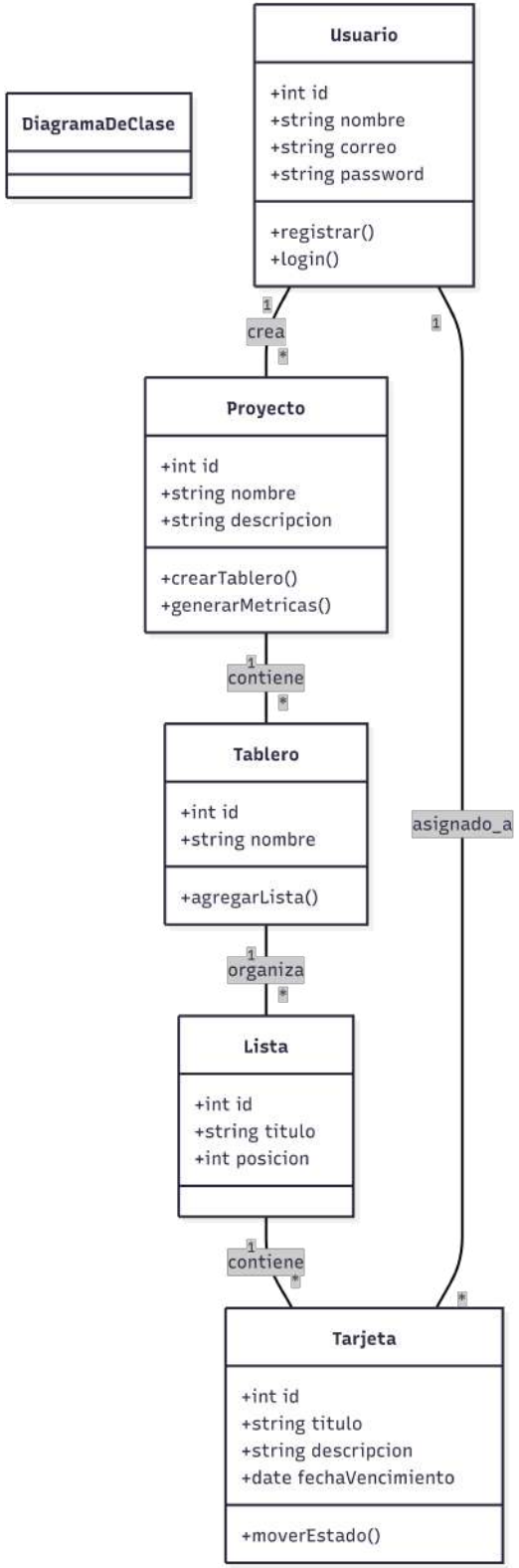

DiagramaDeClase
Usuario
+int id
+string nombre
+string correo
+string password
+registrar()
+login()
1
crea
*
1
Proyecto
+int id
+string nombre
+string descripcion
+crearTablero()
+generarMetricas()
1
contiene
*
Tablero
+int id
+string nombre
+agregarLista()
asignado_a
1
organiza
*
Lista
+int id
+string titulo
+int posicion
1
contiene
*
*
Tarjeta
+int id
+string titulo
+string descripcion
+date fechaVencimiento
+moverEstado()

DIAGRAMA DE COMPONENTES

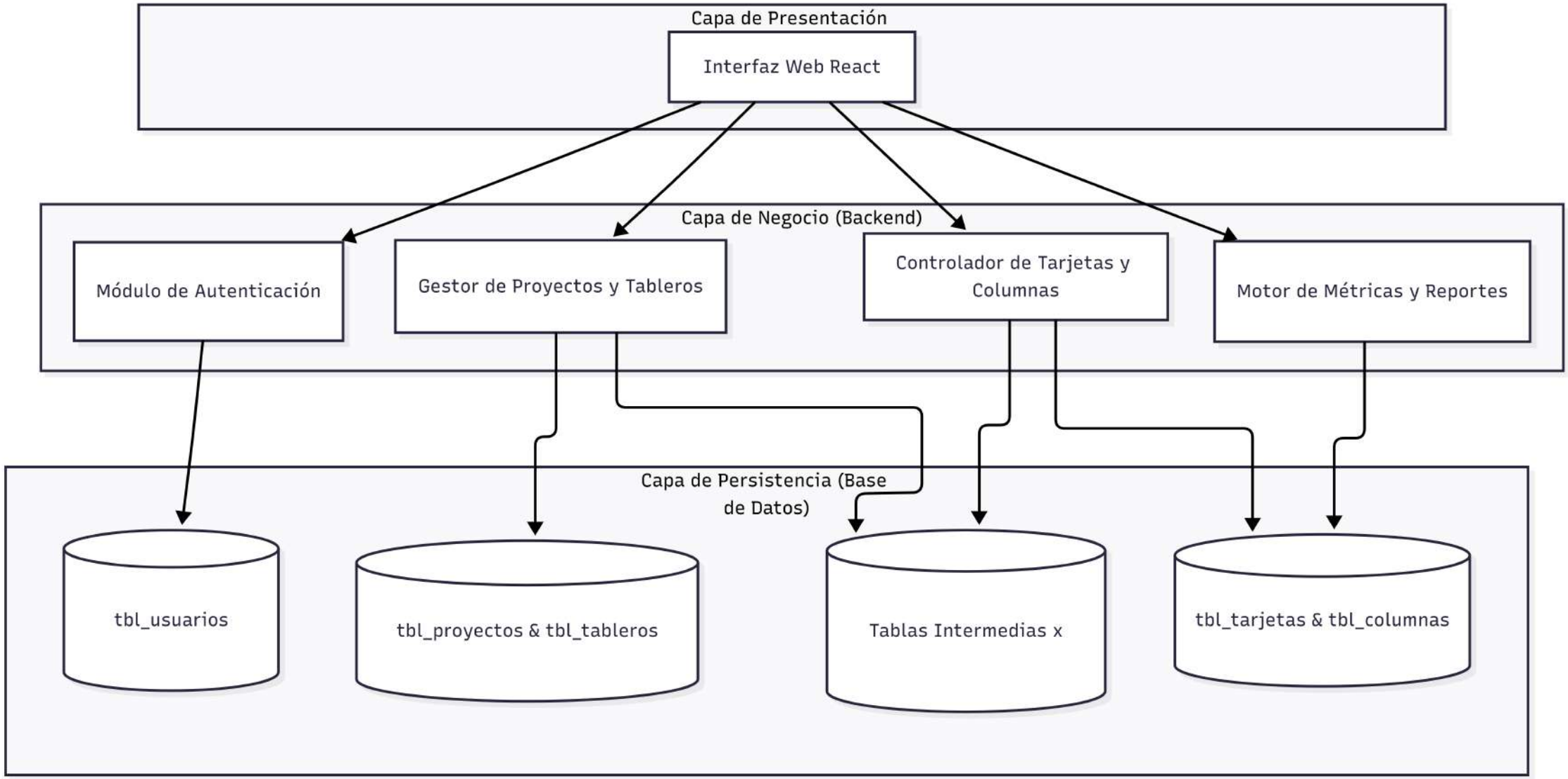
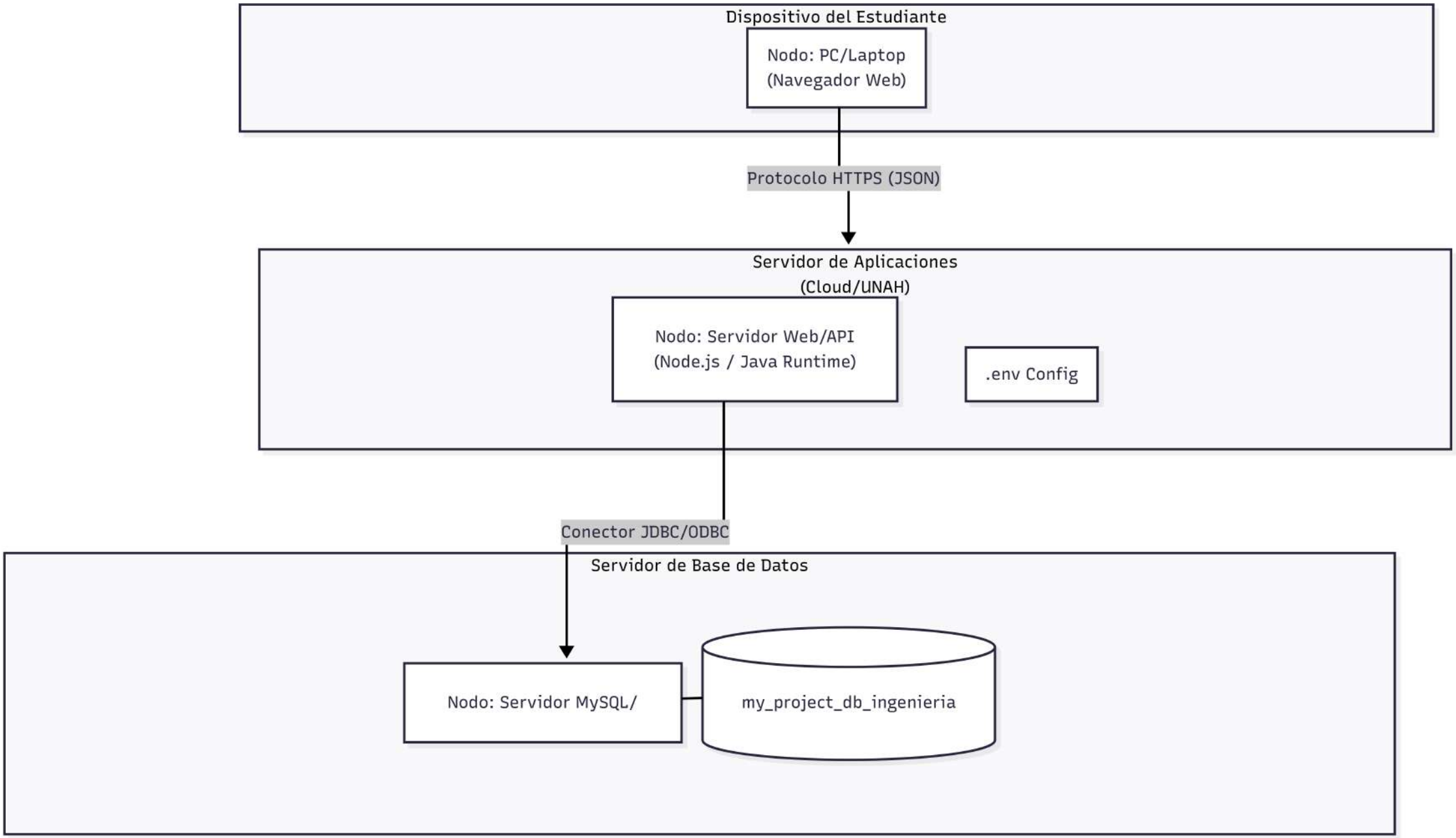
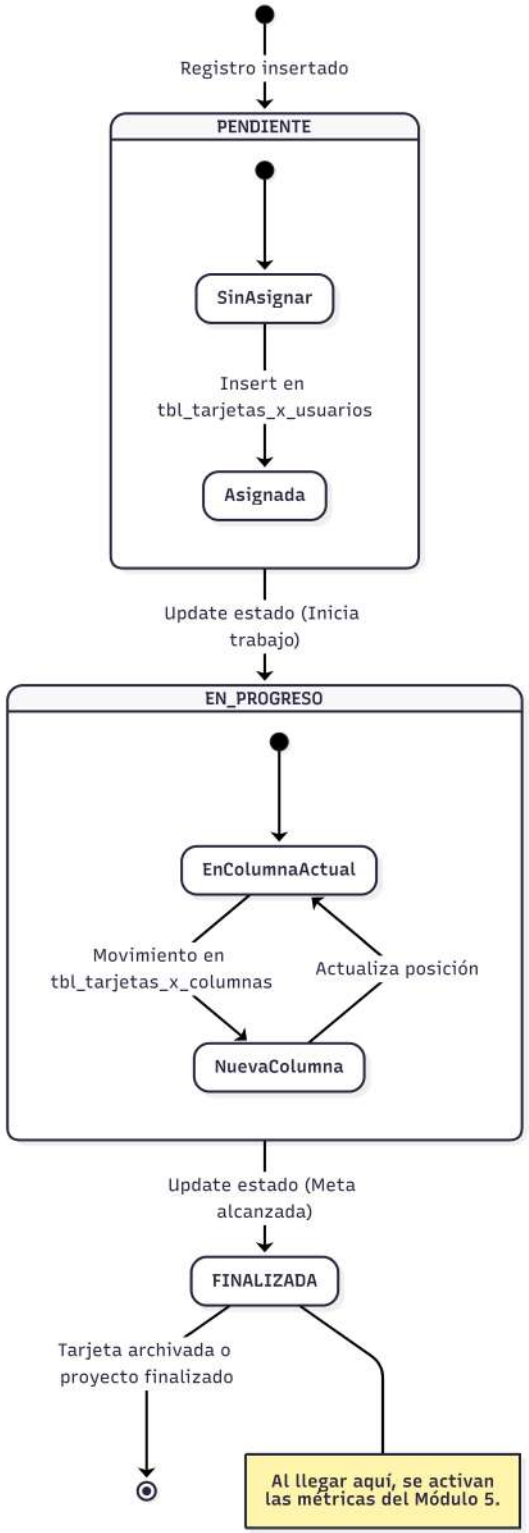
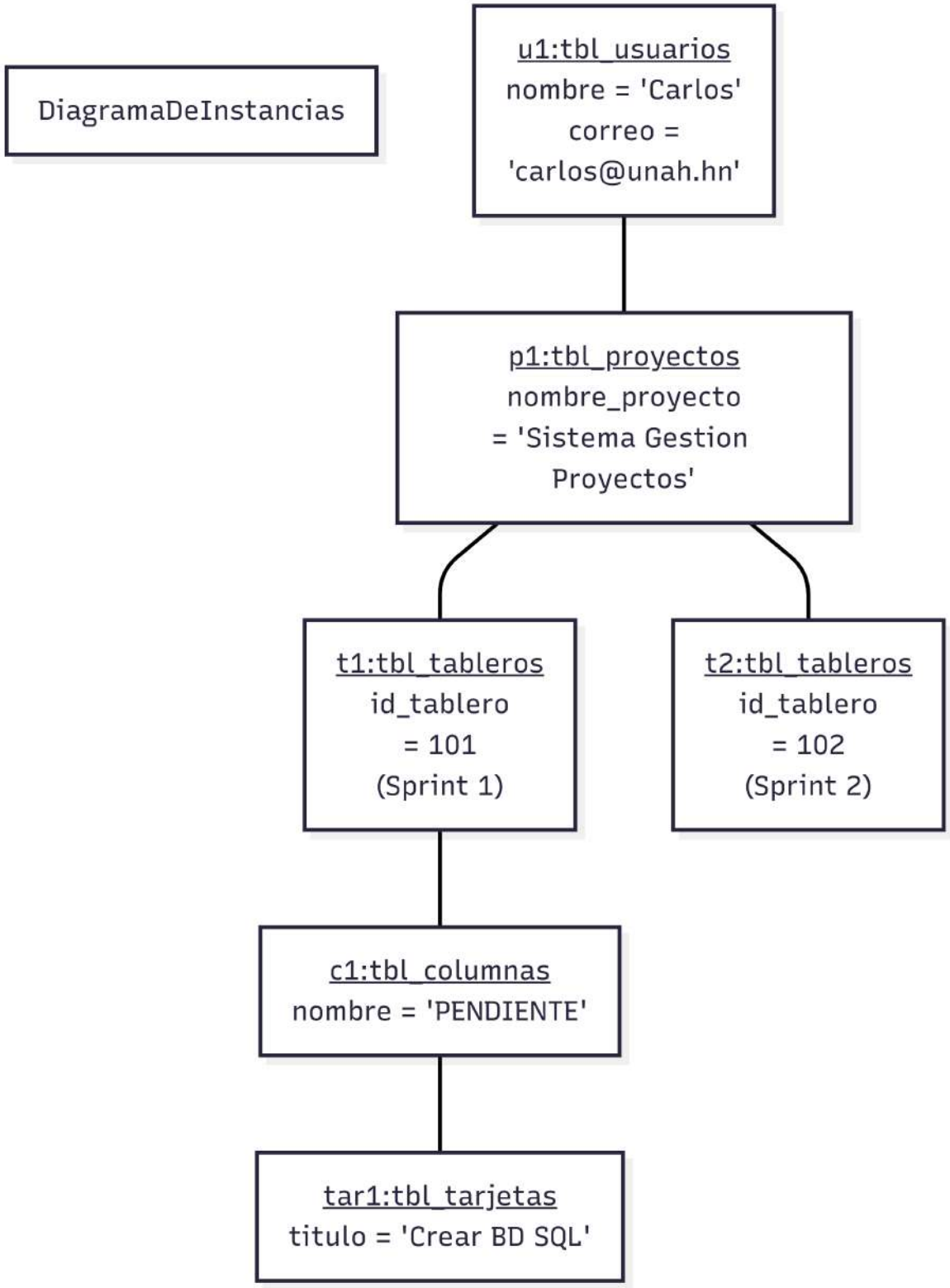
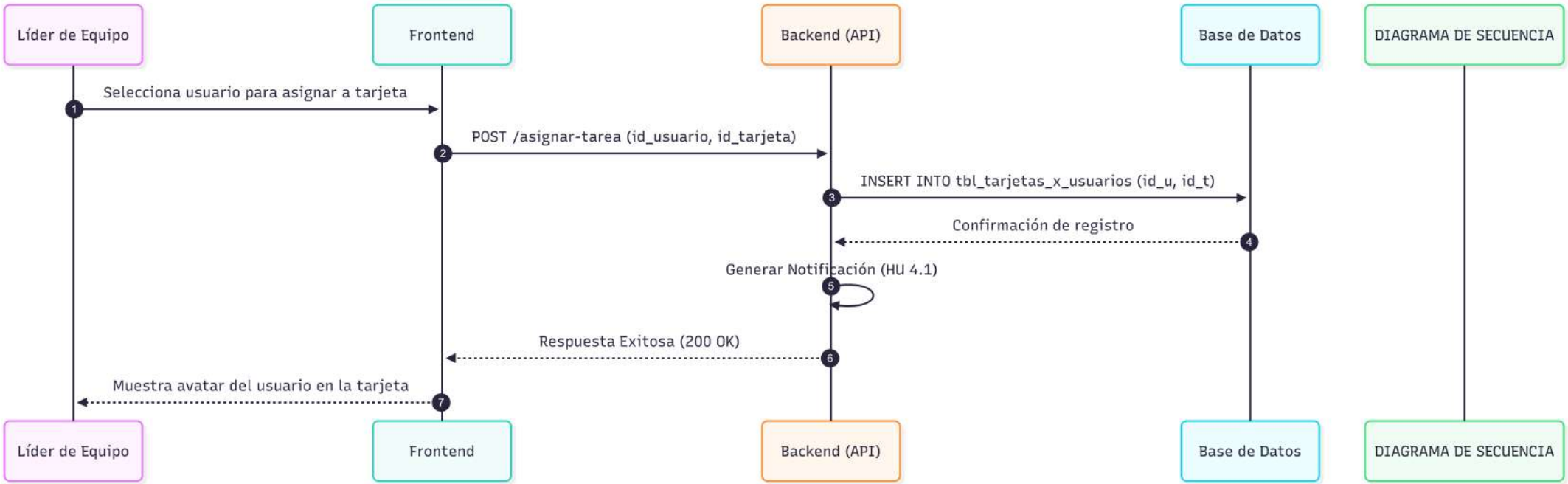


DIAGRAMA DE DESPLIEGUE




Registro insertado
PENDIENTE
SinAsignar
Insert en
tbl_tarjetas_x_usuarios
Asignada
Update estado (Inicia
trabajo)
EN_PROGRESO
EnColumnaActual
Movimiento en
tbl_tarjetas_x_columnas
Actualiza posición
NuevaColumna
Update estado (Meta
alcanzada)
FINALIZADA
Tarjeta archivada o
proyecto finalizado
Al llegar aquí, se activan
las métricas del Módulo 5.


DiagramaDeInstancias
u1:tbl_usuarios
nombre = 'Carlos'
correo =
'carlos@unah.hn'
p1:tbl_proyectos
nombre_proyecto
= 'Sistema Gestion
Proyectos'
t1:tbl_tableros
id_tablero
= 101
(Sprint 1)
t2:tbl_tableros
id_tablero
= 102
(Sprint 2)
c1:tbl_columnas
nombre = 'PENDIENTE'
tar1:tbl_tarjetas
titulo = 'Crear BD SQL'


Líder de Equipo
Frontend
Backend (API)
Base de Datos
DIAGRAMA DE SECUENCIA
1
Selecciona usuario para asignar a tarjeta
2
POST /asignar-tarea (id_usuario, id_tarjeta)
3
INSERT INTO tbl_tarjetas_x_usuarios (id_u, id_t)
4
Confirmación de registro
5
Generar Notificación (HU 4.1)
6
Respuesta Exitosa (200 OK)
7
Muestra avatar del usuario en la tarjeta
Líder de Equipo
Frontend
Backend (API)
Base de Datos
DIAGRAMA DE SECUENCIA